

Foundation of Blockchain Technology (IS501) Pre-Reading Material – Lecture 16

Data Types in Solidity

Estimated Reading Time: 15–20 minutes

Why This Topic Matters

Solidity provides different **data types** for storing and manipulating values in smart contracts. Choosing an appropriate data type is important because it affects **data representation, storage requirements, gas usage, and the range of values** that can be stored.

Learning Outcomes

After completing this pre-reading, you should be able to:

- Identify common Solidity data types.
- Differentiate between **uint**, **int**, and **bool**.
- Understand the purpose of different integer sizes.
- Explain why data type selection matters in blockchain applications.
- Recognize the relationship between data types and **gas optimization**.

1. Basic Solidity Data Types

Some commonly used Solidity data types are:

- **uint** – stores non-negative integers.
- **int** – stores both positive and negative integers.
- **bool** – stores either true or false.

Each variable in Solidity is declared with a **data type** and a **variable name**.

Example:

```
uint count;  
int temperature;  
bool isActive;
```

2. Unsigned Integers — uint

uint is used for **non-negative whole numbers**.

It is commonly used for:

- Counts
- IDs
- Balances
- Quantities

Solidity provides different sizes such as uint8, uint16, uint32, and so on, up to uint256.

The default uint is **uint256**.

Think: Which type would you choose for a variable that can never have a negative value?

3. Signed Integers — int

int can store both **positive and negative whole numbers**.

It is useful when a value may go below zero, such as:

- Temperature changes
- Balance changes
- Signed calculations

Like uint, Solidity provides different sizes such as int8, int16, int32, and up to int256.

The default int is **int256**.

4. Boolean — bool

The bool type stores only two possible values:

- true
- false

It is commonly used for:

- Conditions



- Status indicators
- Flags
- Permissions

Example:

```
bool isActive = true;  
bool isOwner = false;
```

5. Choosing the Appropriate Data Type

Different data types have different **value ranges and storage requirements**.

For example:

- Use uint when negative values are not required.
- Use int when both positive and negative values are possible.
- Use bool when only a true/false condition is required.

Smaller integer types can be useful when the expected values are known to fit within their range.

6. Gas and Storage Considerations

In blockchain applications, data storage has a **gas cost**. Therefore, data types should be selected carefully.

The key principle is:

Choose a data type that can safely represent the required values without unnecessarily increasing storage requirements.

However, students should also consider the **expected value range** before choosing a smaller integer type.

7. Coding Practices

While writing Solidity code:

- Keep variable names meaningful.
- Add comments where necessary.
- Keep code properly formatted.
- Pay attention to compiler or editor warnings.

Prepared by: Dr. Rajesh Kumar

Foundation of Blockchain Technology (IS501) | AY 2026–27

Course Repository: <https://github.com/rajesh101191/Foundation-of-Blockchain-Technology>



- Check that assigned values are compatible with the selected data type.

Think Before the Lecture

1. What is the difference between uint and int?
2. When would you use a bool in a smart contract?
3. Why does Solidity provide different integer sizes?
4. Why is data type selection important in blockchain applications?
5. What could happen if a value exceeds the range supported by its selected integer type?

